5.3 Каналы

Обмен данными и синхронизация горутин в Go.

- Создание и направление канала.
- Буфер, блокировки, закрытие и чтение результатов.
- Ожидание нескольких событий через select и таймауты.

Создание и базовые операции

Независимые фрагменты внутри функции

```
var uninitialized chan int // nil
ch := make(chan int)        // без буфера
buffered := make(chan int, 3)

ch <- 25       // отправка
value := <-ch  // получение
```

- Канал имеет тип элемента: chan int передаёт значения int.
- make создаёт канал, а копирование переменной канала не копирует очередь.
- Операции могут блокироваться: перед отправкой продумайте, кто получит значение.

Небуферизированный канал

Фрагмент main; import "fmt"

```
ch := make(chan string)
go func() {
    ch <- "Hello"
}()
message := <-ch
fmt.Println(message)
```

- Передача требует встречи отправителя и получателя.
- Если получателя пока нет, отправитель ждёт; если нет отправителя, ждёт получатель.
- В примере main дожидается значения прежде, чем завершиться.

Буферизированный канал

Фрагмент функции; import "fmt"

```
ch := make(chan int, 2)
ch <- 10
ch <- 20
fmt.Println(len(ch), cap(ch)) // 2 2
fmt.Println(<-ch)              // 10
ch <- 30                      // место освободилось
```

- Пока есть свободное место, отправителю не нужен немедленный получатель.
- Отправка в заполненный буфер ждёт освобождения места. Получение из пустого канала ждёт значение.
- len - текущий снимок размера очереди. Он не гарантирует, что следующая операция не заблокируется.

Как устроено ожидание

- Модель реализации: буфер, очередь ожидающих отправителей и очередь получателей.
- Для буфера используются текущая длина, ёмкость и позиции чтения/записи.
- В runtime встречаются имена buf, qcount, dataqsiz, sendq, recvq, sendx, recvx и closed.
- Это детали реализации, а не публичный API: прикладной код использует make, отправку, получение и close.
- Безопасность операций канала не делает безопасными любые общие данные, на которые указывают передаваемые значения.

Сигнал завершения нескольких задач

Фрагмент main: ожидание трёх сигналов

```
done := make(chan struct{})
for i := 0; i < 3; i++ {
    go func() {
        // Здесь выполняется конечная задача.
        done <- struct{}{}
    }()
}
for i := 0; i < 3; i++ {
    <-done
}
```

- Пустая структура передаёт факт события, а не полезную нагрузку.
- Ожидание трёх результатов не требует закрывать канал. В реальной задаче передавайте также ошибки.

Направление канала в контракте функции

Две функции

```
func send(out chan<- int) {
    out <- 1
}

func receive(in <-chan int) int {
    return <-in
}
```

- chan<- int разрешает отправку, <-chan int - получение.
- Двунаправленный chan int можно передать функции с нужным ограничением направления.
- Направление проверяет компилятор. Оно не определяет автоматически, кто владеет жизненным циклом канала.

Закрытие: данные и признак ok

Фрагмент функции; import "fmt"

```
ch := make(chan int, 1)
ch <- 7
close(ch)

first, ok1 := <-ch // 7, true: данные буфера
last, ok2 := <-ch  // 0, false: данных больше нет
fmt.Println(first, ok1, last, ok2)
```

- Закрытие сообщает, что новых значений больше не будет.
- Оставшиеся значения буфера доступны для чтения после close.
- Только после опустошения закрытого канала чтение сразу возвращает нулевое значение и false.

range читает до завершения потока

Фрагмент main; import "fmt"

```
ch := make(chan int)
go func() {
    defer close(ch)
    for i := 1; i <= 3; i++ {
        ch <- i
    }
}()
for value := range ch {
    fmt.Println(value)
}
```

- range завершится после закрытия и чтения всех накопленных значений.
- Если отправитель перестал писать, но не закрыл канал, цикл может ждать бесконечно.

Кто закрывает канал

- Закрывает сторона, которая знает, что отправок больше не будет. Обычно это отправитель или координатор отправителей.
- Если отправителей несколько, координатор сначала дожидается их завершения и лишь затем закрывает канал.
- Отправка в закрытый канал и повторный close вызывают panic.
- Получатель не должен закрывать канал, пока отправители могут им пользоваться.
- Каналы не обязательно закрывать ради освобождения памяти; close нужен для протокола взаимодействия.

select: несколько возможных событий

Фрагмент функции; first и second - каналы

```
select {
case value := <-first:
    fmt.Println("first:", value)
case value := <-second:
    fmt.Println("second:", value)
}
```

- Если готова одна операция, выполняется она. Если готовы несколько, выбирается одна псевдослучайно.
- Порядок веток не задаёт приоритет.
- Если ни одна операция не готова и default нет, select блокируется.

default: проверка без ожидания

Фрагмент функции; import "fmt"

```
select {
case value := <-ch:
    fmt.Println(value)
default:
    fmt.Println("Сейчас данных нет")
}
```

- default выбирается только когда ни одна операция сейчас не готова.
- Это не гарантирует, что данные не появятся через мгновение.
- Бесконечный цикл с таким select без паузы или другой работы может зря расходовать CPU.

Таймаут получения и отправки

Независимые фрагменты; импорты fmt и time

```
select {
case value := <-result:
    fmt.Println(value)
case <-time.After(2 * time.Second):
    fmt.Println("Таймаут получения")
}

select {
case out <- 42:
    fmt.Println("Отправлено")
case <-time.After(time.Second):
    fmt.Println("Таймаут отправки")
}
```

Таймаут прекращает это ожидание, но не отменяет вычисление отправителя. Для отмены согласуйте отдельный сигнал или context.

nil, закрытый канал и блокировки

- nil: отправка и получение блокируются; close(nil) вызывает panic.
- Открытый канал: блокировка зависит от наличия партнёра и места в буфере.
- Закрытый канал: отправка вызывает panic; получение после чтения буфера готово сразу.
- В select ветку с nil-каналом можно использовать как временно отключённую.
- Закрытый канал постоянно готов к получению. В цикле проверяйте ok и убирайте завершённый канал из ожидания.

Deadlock: нет пути продолжения

Намеренно ошибочный пример

```
ch := make(chan int)
ch <- 5 // в одной main: получателя нет
```

- Операция без подходящего партнёра ждёт; сама по себе одна заблокированная горутина ещё не означает deadlock всей программы.
- Если программа в целом не может продолжиться, runtime может сообщить о deadlock.
- Для передачи запустите получателя параллельно или используйте подходящий буфер. Размер буфера не заменяет корректный протокол.

Практическая проверка

- У каждой отправки должен быть ожидаемый получатель или обоснованный запас буфера.
- У потока данных должен быть понятный сигнал завершения.
- Используйте value, ok или range, чтобы отличать конец потока от нулевого значения.
- При остановке получателя предусмотрите отмену отправителя, чтобы он не остался ждать.
- Проверьте обычный сценарий, пустой поток, закрытие, ошибку и таймаут.

Материалы и источники

Учебная реконструкция по сохранившемуся конспекту урока 5.3 «Каналы». Уточнены семантика close/ok, выбор ветки select и отличие блокировки операции от deadlock программы. Утраченный оригинальный PDF не найден.

https://go.dev/ref/spec#Channel_types
https://go.dev/ref/spec#Send_statements
https://go.dev/ref/spec#Receive_operator
https://go.dev/ref/spec#Select_statements
https://go.dev/doc/effective_go#channels